Code Generation

Three primary tasks:
1. Instruction selection
2. Register allocation & assignment
3. Instruction ordering

Issues
1. Input
2. Target Program
3.

1. Input

Input = IR produced by frontend + info in symbol table [determines runtime addresses of data objects denoted by names in IR]

many choices
- TAC: quadruples, triples, indirect triples
- VM representation: bytecode, stack machine code
- linear: postfix notation
- graphical: syntax trees, DAGs

Assumptions:
- → frontend has scanned, parsed and translated source into relatively low-level IR
- → All syntactic and static semantic errors detected

2. Target Program

What instruction set will you use? Significant impact on difficulty of implementation

RISC	CISC	Stack-based
• One clk for every instr		• Revived w/ introduction of JVM
• More regs		
• 3 address	2 address	
• Simple addressing modes	Variety	
• Simple instr-set	Variable-length instr	

Target program → absolute / relocatable

3. Instruction Selection

How is the mapping created from IR to target?

Complexity depends on:
1. level of IR
2. Nature of ISA
3. Desired quality of generated code

(a) level of IR
- IR instr high level → sequence of machine instr using templates → poor code that needs further optimization
- " " low level

(b) Nature of ISA
Uniformity and completeness

(c) Desired quality
Do not care about efficiency → instruction selection straightforward

```
a = b+c          LD R0, b
d = a+e    →     ADD R0, R0, c
                 ST a, R0      } redundant
                 LD R0, a      }
                 ADD R0, R0, e
                 ST R0, d
```

4. Register Allocation

Register fastest but usually not enough of them to hold all values
↓
So what do we hold in them?

Eg:
```
i = 0
s = 0
L1: if i>=n goto L2
    s = s+i
    i = i+1
L2: goto L1
```

What if you have procedures?

```
int func (int param) {
    int i = 1
    ...
    return param
}
```
param → passing args
i → local vars

Code Generator Algorithm

Algorithm getReg()

Input: Request for reg Output: Register/memory location

1. Return a new register if available
2. If value of y stored in R, R holds only y AND y has no next use,
   return R
   update address into address descriptor
3. Else no free registers
4.

Four principal uses of registers
1. some or all operands should be in registers for most ops
2. good temporary for local block scoped vars
3. used to hold computed global values used by other blocks
4.

Eg: x = y+z
z = x*z
y = z
x = y+z

Instr	Register R1	Register R2	Address x	Address y	Address z
	—	—			
LOAD R1, y	y				
LOAD R2, z	y	z			
ADD R2, R1, R2	y	x			
MUL R1, R2, R2	z	x			
ST y, R2	z, y	x			
ADD R2, R1, R1	z, y	x			
exit ST x, R1					
ST z, R2					